//placement

```
#include<iostream>
using namespace std;
int main()
{   int block[50],process[50],allocation[50],b,p;
   cout<<"Enter number of blocks: "; cin>>b;
   cout<<"Enter size of blocks: "<<endl;
   for(int i=0;i<b;i++) cin>>block[i];
   cout<<"Enter number of processes: "; cin>>p;
   cout<<"Enter size of processes: "<<endl;
   for(int i=0;i<p;i++) cin>>process[i];
   // FIRST FIT
   cout<<"\nFirst Fit Allocation\n";
   int tempBlock1[50];
   for(int i=0;i<b;i++) tempBlock1[i]=block[i];
   for(int i=0;i<p;i++)
   {       allocation[i]=-1;
      for(int j=0;j<b;j++)
      {    if(tempBlock1[j]>=process[i])
         {       allocation[i]=j;
            tempBlock1[j]-=process[i];
            break;}  }
      if(allocation[i]!=-1) cout<<"Process "<<i+1<<" -> Block "<<allocation[i]+1<<endl;
      else cout<<"Process "<<i+1<<" -> Not Allocated"<<endl; }
   // BEST FIT
   cout<<"\nBest Fit Allocation\n";
   int tempBlock2[50];
   for(int i=0;i<b;i++) tempBlock2[i]=block[i];
   for(int i=0;i<p;i++)
   {       allocation[i]=-1;
      int bestIndex=-1;
      for(int j=0;j<b;j++)
      {     if(tempBlock2[j]>=process[i])
         {       if(bestIndex==-1 || tempBlock2[j]<tempBlock2[bestIndex]) bestIndex=j;
         }       }
      if(bestIndex!=-1)
      {          allocation[i]=bestIndex;
         tempBlock2[bestIndex]-=process[i];
         cout<<"Process "<<i+1<<" -> Block "<<bestIndex+1<<endl;
      }       else cout<<"Process "<<i+1<<" -> Not Allocated"<<endl;}
   // WORST FIT
   cout<<"\nWorst Fit Allocation\n";
  int tempBlock3[50];
   for(int i=0;i<b;i++) tempBlock3[i]=block[i];
   for(int i=0;i<p;i++)
   {       allocation[i]=-1;    int worstIndex=-1;
      for(int j=0;j<b;j++)    {
         if(tempBlock3[j]>=process[i]){
            if(worstIndex==-1 || tempBlock3[j]>tempBlock3[worstIndex]) worstIndex=j;
         }  }
      if(worstIndex!=-1)  {
         allocation[i]=worstIndex; tempBlock3[worstIndex]-=process[i];
         cout<<"Process "<<i+1<<" -> Block "<<worstIndex+1<<endl;
      }    else cout<<"Process "<<i+1<<" -> Not Allocated"<<endl; }
   // NEXT FIT
   cout<<"\nNext Fit Allocation\n"; int tempBlock4[50];
   for(int i=0;i<b;i++) tempBlock4[i]=block[i];
   int lastAllocated=0;
   for(int i=0;i<p;i++)
   {   allocation[i]=-1;
      int count=0,j=lastAllocated;
      while(count<b)
      {    if(tempBlock4[j]>=process[i])
         {  allocation[i]=j;
            tempBlock4[j]-=process[i];
            lastAllocated=j;
            break; }
         j=(j+1)%b;       count++;       }
      if(allocation[i]!=-1) cout<<"Process "<<i+1<<" -> Block "<<allocation[i]+1<<endl;
      else cout<<"Process "<<i+1<<" -> Not Allocated"<<endl;    }    return 0;}
```

//bankers

```
#include <iostream>
using namespace std;
int main() {
  int n, m;
  //resouces and process input
  cout << "Enter number of processes: ";
  cin >> n;
  cout << "Enter number of resources: ";
  cin >> m;
  //matrix of alloc max & available
  int alloc[n][m], max[n][m], avail[m];
  //alocation matrix
  cout << "\nEnter Allocation Matrix:\n";
  for(int i = 0; i < n; i++)
   for(int j = 0; j < m; j++)
    cin >> alloc[i][j];
  //max
  cout << "\nEnter Max Matrix:\n";
  for(int i = 0; i < n; i++)
   for(int j = 0; j < m; j++)
    cin >> max[i][j];
  //availavble
  cout << "\nEnter Available Resources:\n";
  for(int j = 0; j < m; j++)
   cin >> avail[j];
  //need matrix
  int need[n][m];
  for(int i = 0; i < n; i++)
{   for(int j = 0; j < m; j++)
  need[i][j] = max[i][j] - alloc[i][j]; }
for(int j=0;j<m;j++)
  cout << need[i][j] << " ";
cout << endl; }
  bool finish[n] = {false}; //to check if process is complete
  int safeSeq[n]; //safesequence storage
  int count = 0; //to count no of completed process
  //loop runs for all process-> complete
  while(count < n) {
    bool found = false; //to check proess not complete
    for(int i = 0; i < n; i++) {
      if(!finish[i]) {
        int j;
        for(j = 0; j < m; j++) {
          if(need[i][j] > avail[j])
            break;
        }
        if(j == m) {   //all resources r satisfied
          //to rerelease resources
          for(int k = 0; k < m; k++)
            avail[k] += alloc[i][k];
          //to add process to safe sequence
          safeSeq[count++] = i;
          finish[i] = true;
          found = true;
        }     }   }
    // deadlock possibility
    if(!found) {
      cout << "\nSystem is NOT in safe state (Deadlock possible)\n";
      return 0;   } }
  // safe sequence possibility
  cout << "\nSystem is in SAFE state.\nSafe Sequence: ";
  for(int i = 0; i < n; i++)
    cout << "P" << safeSeq[i] << " ";
  cout << endl;  return 0; }
```

//deadlock

```
#include <iostream>
using namespace std;
int main(){
int n, m;
cout << "Enter number of processes: ";
  cin >> n;
  cout << "Enter number of resource types: ";
  cin >> m;
  int allocation[n][m], request[n][m], available[m];
  cout << "\nEnter Allocation Matrix:\n";
  for(int i = 0; i < n; i++)
  {    for(int j = 0; j < m; j++)
    { cin >> allocation[i][j];  } }
  cout << "\nEnter Request Matrix:\n";
  for(int i = 0; i < n; i++)
  { for(int j = 0; j < m; j++)
    {  cin >> request[i][j];  } }
  cout << "\nEnter Available Resources:\n";
  for(int i = 0; i < m; i++)
  { cin >> available[i]; }
  int work[m] , bool finish[n];
  // Step 1: Initialize
  for(int i = 0; i < m; i++)
  { work[i] = available[i]; }
  for(int i = 0; i < n; i++)
  { finish[i] = false; }
  // Step 2 & 3
  bool found;
  do  {
    found = false;
    for(int i = 0; i < n; i++)
    { if(finish[i] == false)
      { bool possible = true;
        for(int j = 0; j < m; j++)
        { if(request[i][j] > work[j])
          { possible = false;
            break; } }
        if(possible)
        { for(int j = 0; j < m; j++)
          { work[j] += allocation[i][j]; }
          finish[i] = true;
          found = true; } }  } }
while(found);
  // Step 4: Check deadlock
  bool deadlock = false;
  cout << "\nDeadlocked Processes: ";
  for(int i = 0; i < n; i++)
  {if(finish[i] == false)
    { cout << "P" << i << " ";
      deadlock = true; } }
  if(deadlock == false)
  { cout << "No Deadlock"; }
  return 0;}
```

//buddy

```
#include <iostream>
#include <cmath>
using namespace std;
struct Block {
  int size;
  bool free;
};
int main() {
  int memorySize, n;
  cout << "Enter total memory size (power of 2): ";
  cin >> memorySize;
  cout << "Enter number of processes: ";
  cin >> n;
  int process[n];
  cout << "Enter process sizes:\n";
  for(int i = 0; i < n; i++)
    cin >> process[i];
  Block blocks[100];
  // initially one free block
  int blockCount = 1;
  blocks[0].size = memorySize;
  blocks[0].free = true;
  for(int i = 0; i < n; i++) {
    int required = 1;
    // find nearest power of 2
    while(required < process[i])
      required *= 2;
    bool allocated = false;
    for(int j = 0; j < blockCount; j++) {
      // split until suitable size
      while(blocks[j].size / 2 >= required && blocks[j].free) {
        blocks[blockCount].size = blocks[j].size / 2;
        blocks[blockCount].free = true;
        blocks[j].size /= 2;
        blockCount++; }
      // allocate block
      if(blocks[j].free && blocks[j].size >= required) {
        blocks[j].free = false;
        cout << "\nProcess " << i + 1
          << " of size " << process[i]
          << " allocated in block size "
          << blocks[j].size;
        cout << "\nInternal Fragmentation = "
          << blocks[j].size - process[i] << endl;
        allocated = true;
        break;  }  }
    if(!allocated) {
      cout << "\nProcess " << i + 1
        << " cannot be allocated.\n";   } }
  // memory status
  cout << "\n\nMemory Blocks Status:\n";
  for(int i = 0; i < blockCount; i++) {
    cout << "Block " << i + 1
      << " Size: " << blocks[i].size
      << " -> ";
    if(blocks[i].free)
      cout << "Free";
    else
      cout << "Allocated";
    cout << endl; }
  return 0;}
```

/ //lru

```
#include<iostream>
using namespace std;
int main()
{ int n,f,p[50],fr[10],time[10],pf=0,cnt=0;
  cin>>n;
  for(int i=0;i<n;i++) cin>>p[i];
  cin>>f;
  for(int i=0;i<f;i++)
  { fr[i]=-1;
    time[i]=0; }
  for(int i=0;i<n;i++)
  { bool found=false;
    for(int j=0;j<f;j++)
    { if(fr[j]==p[i])
      { cnt++;
        time[j]=cnt;
        found=true;  } }
    if(!found)
    { int pos=0;
      for(int j=1;j<f;j++)
        if(time[j]<time[pos]) pos=j;
      cnt++;
      fr[pos]=p[i];
      time[pos]=cnt;
      pf++;  }  }
  cout<<"Page Faults = "<<pf;
  return 0; }
```

//optimal

```
#include<iostream>
using namespace std;
int main()
{ int n,f,p[50],fr[10],pf=0;
  cin>>n;
  for(int i=0;i<n;i++) cin>>p[i];
  cin>>f;
  for(int i=0;i<f;i++) fr[i]=-1;
  for(int i=0;i<n;i++)
  {    bool found=false;
    for(int j=0;j<f;j++)
      if(fr[j]==p[i]) found=true;
    if(!found)
    { int pos=0,far=-1;
      for(int j=0;j<f;j++)
      { if(fr[j]==-1)
        {pos=j;
          break;   }
        int k;
        for(k=i+1;k<n;k++)
          if(fr[j]==p[k]) break;
        if(k>far)
        { far=k;
          pos=j; } }
      fr[pos]=p[i];
      pf++; } }
  cout<<"Page Faults = "<<pf;
  return 0; }
```

//fifo

```
#include <iostream>
using namespace std;
int main(){
  int pages[50], frame[10];
  int n, f, pageFault = 0;
  cout << "Enter number of pages: ";
  cin >> n;
  cout << "Enter page reference string:\n";
  for(int i = 0; i < n; i++)
  cin >> pages[i];
  cout << "Enter number of frames: ";
  cin >> f;
  // initialize frames with -1
  for(int i = 0; i < f; i++)
  frame[i] = -1;
  int index = 0;
  for(int i = 0; i < n; i++)
  { bool found = false;
  // check if page already exists
  for(int j = 0; j < f; j++)
  { if(frame[j] == pages[i])
  { found = true; break; } }
  // page fault
  if(!found) {
  frame[index] = pages[i];
  index = (index + 1) % f;
  pageFault++;
  cout << "Frames: ";
  for(int j = 0; j < f; j++)
  cout << frame[j] << " ";
  cout << endl; } }
  cout << "\nTotal Page Faults = " << pageFault;
  return 0; }
```

//logical ti physical

```
#include <iostream>
using namespace std;
int main()
{  int logicalAddress, pageSize, numPages;
  cout << "Enter page size: ";
  cin >> pageSize;
  cout << "Enter number of pages: ";
  cin >> numPages;
  int pageTable[numPages];
  cout << "Enter page table (Frame numbers):\n";
  for(int i = 0; i < numPages; i++)
  { cout << "Page " << i << " -> Frame: ";
    cin >> pageTable[i];  }
  cout << "Enter logical address: ";
  cin >> logicalAddress;
  int pageNumber = logicalAddress / pageSize;
  int offset = logicalAddress % pageSize;
  if(pageNumber >= numPages)
  { cout << "Invalid Logical Address!";}
  else {
    int frameNumber = pageTable[pageNumber];
    int physicalAddress = frameNumber * pageSize + offset;
    cout << "\nPage Number: " << pageNumber;
    cout << "\nOffset: " << offset;
    cout << "\nFrame Number: " << frameNumber;
    cout << "\nPhysical Address: " << physicalAddress;
  }  return 0; }
```

//ldisk

```
//fifo
#include <iostream>
#include <cmath>
using namespace std;
int main() {
  int n, head;
    cout << "Enter number of requests: ";
  cin >> n;
  int req[n];
  cout << "Enter request sequence:\n";
  for(int i = 0; i < n; i++)
    cin >> req[i];
  cout << "Enter initial head position: ";
  cin >> head;
  int seek = 0;
  for(int i = 0; i < n; i++) {
    seek += abs(head - req[i]);
    head = req[i];  }
  cout << "Total Seek Time (FCFS): " << seek;
  return 0; }

//sstf
#include <iostream>
#include <cmath>
using namespace std;
int main() {
  int n, head;
  cout << "Enter number of requests: ";
  cin >> n;
  int req[n], completed[n];
  cout << "Enter request sequence:\n";
  for(int i = 0; i < n; i++) {
    cin >> req[i];
    completed[i] = 0;  }
  cout << "Enter initial head position: ";
  cin >> head;
  int seek = 0;
  for(int i = 0; i < n; i++) {
    int min = 1e9, index = -1;
    for(int j = 0; j < n; j++) {
      if(!completed[j]) {
        int dist = abs(head - req[j]);
        if(dist < min) {
          min = dist;
          index = j; } } }
    seek += min;
    head = req[index];
    completed[index] = 1;  }
  cout << "Total Seek Time (SSTF): " << seek;
  return 0; }
```